

CSE 3204
Formal Language, Automata and Computability



Lecture 27
**Decision Properties of Regular Language, Homomorphism and
Inverse Homomorphism**

Testing Emptiness of a Regular Language

- Representation is any kind of finite automaton:
 - Is there any path from a start state to some accepting state?
 - A graph reachability problem.
 - Time complexity is $O(n^2)$ where n is the number of states.

BASIS: The start state is surely reachable from the start state.

INDUCTION: If state q is reachable from the start state, and there is an arc from q to p with any label (an input symbol, or ϵ if the automaton is an ϵ -NFA), then p is reachable.

Testing Emptiness of a Regular Language

- Representation is a regular expression:
 - Convert the expression into ϵ -NFA.
 - the automaton from the regular expression of length n has at most $O(n)$ states and $O(n)$ transitions. The graph reachability problem will take $O(n)$ time.
 - Inspect the regular expression to decide whether it is empty.

BASIS: $\emptyset$ denotes the empty language; ϵ and a for any input symbol a do not.

INDUCTION: Suppose R is a regular expression. There are four cases to consider, corresponding to the ways that R could be constructed.

1. $R = R_1 + R_2$. Then $L(R)$ is empty if and only if both $L(R_1)$ and $L(R_2)$ are empty.
2. $R = R_1 R_2$. Then $L(R)$ is empty if and only if either $L(R_1)$ or $L(R_2)$ is empty.
3. $R = R_1^*$. Then $L(R)$ is not empty; it always includes at least ϵ .
4. $R = (R_1)$. Then $L(R)$ is empty if and only if $L(R_1)$ is empty, since they are the same language.

Testing Membership in a Regular Language

- Given a string w and a regular language L , whether w is in L ?
- If L is represented by a DFA:
 - Process w in the DFA.
 - If $|w| = n$, then the test will require $O(n)$ time.
- If L is represented by a NFA:
 - Process w in the NFA.
 - If w is of length n , and the NFA has s states, then the running time of this algorithm is $O(ns^2)$.
- If L is represented by a ϵ -NFA:
 - Process w in the ϵ -NFA.
 - First, take the previous set of states and find their successors on input symbol a . Next, compute the ϵ -closure of this set of states.
 - Both steps require $O(s^2)$ time.
 - The running time of this algorithm is $O(ns^2)$.

Testing Membership in a Regular Language

- If L is a regular expression of size s ,
 - convert to an ϵ -NFA with at most $2s$ states $O(s)$ time
 - Perform simulation.
 - The running time of the simulation is **$O(ns^2)$** .

Homomorphism

Formally, if h is a homomorphism on alphabet Σ , and $w = a_1a_2\cdots a_n$ is a string of symbols in Σ , then $h(w) = h(a_1)h(a_2)\cdots h(a_n)$. That is, we apply h to each symbol of w and concatenate the results, in order. For in-

Further, we can apply a homomorphism to a language by applying it to each of the strings in the language. That is, if L is a language over alphabet Σ , and h is a homomorphism on Σ , then $h(L) = \{h(w) \mid w \text{ is in } L\}$. For instance, if L is the language of regular expression 10^*1 , i.e., any number of 0's surrounded by single 1's, then $h(L)$ is the language $(ab)^*$. The reason is

Example 4.13: The function h defined by $h(0) = ab$ and $h(1) = \epsilon$ is a homomorphism. Given any string of 0's and 1's, it replaces all 0's by the string ab and replaces all 1's by the empty string. For example, h applied to the string 0011 is $abab$. $\square$

Homomorphism

Theorem 4.14: If L is a regular language over alphabet Σ , and h is a homomorphism on Σ , then $h(L)$ is also regular.

PROOF: Let $L = L(R)$ for some regular expression R . In general, if E is a regular expression with symbols in Σ , let $h(E)$ be the expression we obtain by replacing each symbol a of Σ in E by $h(a)$. We claim that $h(R)$ defines the language $h(L)$.

The proof is an easy structural induction that says whenever we take a subexpression E of R and apply h to it to get $h(E)$, the language of $h(E)$ is the same language we get if we apply h to the language $L(E)$. Formally, $L(h(E)) = h(L(E))$.

BASIS: If E is ϵ or $\emptyset$, then $h(E)$ is the same as E , since h does not affect the string ϵ or the language $\emptyset$. Thus, $L(h(E)) = L(E)$. However, if E is $\emptyset$ or ϵ , then $L(E)$ contains either no strings or a string with no symbols, respectively. Thus $h(L(E)) = L(E)$ in either case. We conclude $L(h(E)) = L(E) = h(L(E))$.

The only other basis case is if $E = \mathbf{a}$ for some symbol a in Σ . In this case, $L(E) = \{a\}$, so $h(L(E)) = \{h(a)\}$. Also, $h(E)$ is the regular expression that is the string of symbols $h(a)$. Thus, $L(h(E))$ is also $\{h(a)\}$, and we conclude $L(h(E)) = h(L(E))$.

INDUCTION: There are three cases, each of them simple. We shall prove only the union case, where $E = F + G$. The way we apply homomorphisms to regular expressions assures us that $h(E) = h(F + G) = h(F) + h(G)$. We also know that $L(E) = L(F) \cup L(G)$ and

$$L(h(E)) = L(h(F) + h(G)) = L(h(F)) \cup L(h(G)) \quad (4.2)$$

by the definition of what “+” means in regular expressions. Finally,

$$h(L(E)) = h(L(F) \cup L(G)) = h(L(F)) \cup h(L(G)) \quad (4.3)$$

because h is applied to a language by application to each of its strings individually. Now we may invoke the inductive hypothesis to assert that $L(h(F)) = h(L(F))$ and $L(h(G)) = h(L(G))$. Thus, the final expressions in (4.2) and

(4.3) are equivalent, and therefore so are their respective first terms; that is, $L(h(E)) = h(L(E))$.

Inverse Homomorphism

preserve regular languages. That is, suppose h is a homomorphism from some alphabet Σ to strings in another (possibly the same) alphabet T .² Let L be a language over alphabet T . Then $h^{-1}(L)$, read “ h inverse of L ,” is the set of strings w in Σ^* such that $h(w)$ is in L . Figure 4.5 suggests the effect of

Example 4.15: Let L be the language of regular expression $(00 + 1)^*$. That is, L consists of all strings of 0’s and 1’s such that all the 0’s occur in adjacent pairs. Thus, 0010011 and 10000111 are in L , but 000 and 10100 are not.

3. If w has two consecutive a ’s, then $h(w)$ has a substring 0101. Here too, there is an isolated 0 in w .
4. Likewise, if w has two consecutive b ’s, then $h(w)$ has substring 1010 and has an isolated 0.

Thus, whenever one of the above cases hold, $h(w)$ is not in L . However, unless at least one of items (1) through (4) hold, then w is of the form $baba \cdots ba$. To see why, assume none of (1) through (4) hold. Then (1) tells us w must begin with b , and (2) tells us w ends with a . Statements (3) and (4) tell us that a ’s and b ’s must alternate in w . Thus, the logical “OR” of (1) through (4) is equivalent to the statement “ w is not of the form $baba \cdots ba$.” We have proved that the “OR” of (1) through (4) implies $h(w)$ is not in L . That statement is the contrapositive of the statement we wanted: “if $h(w)$ is in L , then w is of the form $baba \cdots ba$.” $\square$

Let h be the homomorphism defined by $h(a) = 01$ and $h(b) = 10$. We claim that $h^{-1}(L)$ is the language of regular expression $(ba)^*$, that is, all strings of repeating ba pairs. We shall prove that $h(w)$ is in L if and only if w is of the form $baba \cdots ba$.

(If) Suppose w is n repetitions of ba for some $n \geq 0$. Note that $h(ba) = 1001$, so $h(w)$ is n repetitions of 1001. Since 1001 is composed of two 1’s and a pair of 0’s, we know that 1001 is in L . Therefore any repetition of 1001 is also formed from 1 and 00 segments and is in L . Thus, $h(w)$ is in L .

(Only-if) Now, we must assume that $h(w)$ is in L and show that w is of the form $baba \cdots ba$. There are four conditions under which a string is *not* of that form, and we shall show that if any of them hold then $h(w)$ is not in L . That is, we prove the contrapositive of the statement we set out to prove.

1. If w begins with a , then $h(w)$ begins with 01. It therefore has an isolated 0, and is not in L .
2. If w ends in b , then $h(w)$ ends in 10, and again there is an isolated 0 in $h(w)$.

Inverse Homomorphism

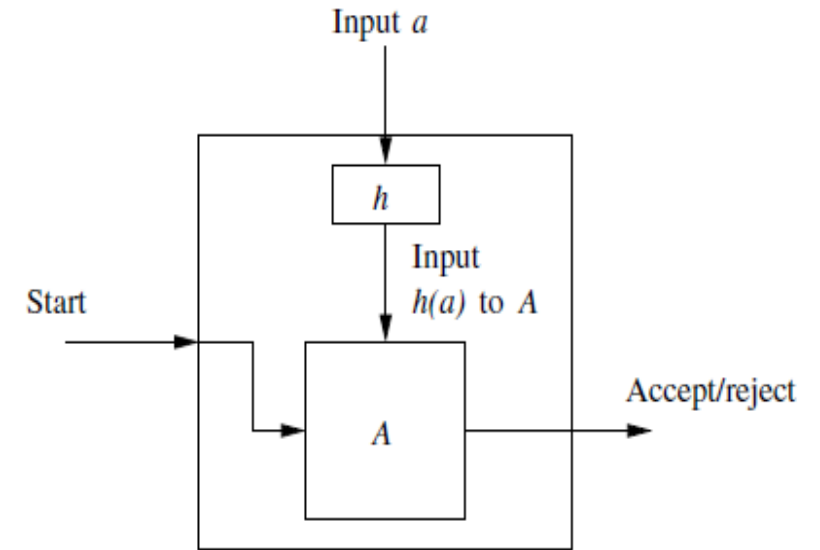
Theorem 4.16: If h is a homomorphism from alphabet Σ to alphabet T , and L is a regular language over T , then $h^{-1}(L)$ is also a regular language.

PROOF: The proof starts with a DFA A for L . We construct from A and h a DFA for $h^{-1}(L)$ using the plan suggested by Fig. 4.6. This DFA uses the states of A but translates the input symbol according to h before deciding on the next state.

Formally, let L be $L(A)$, where DFA $A = (Q, T, \delta, q_0, F)$. Define a DFA

$$B = (Q, \Sigma, \gamma, q_0, F)$$

where transition function γ is constructed by the rule $\gamma(q, a) = \delta(q, h(a))$. That is, the transition B makes on input a is the result of the sequence of transitions that A makes on the string of symbols $h(a)$. Remember that $h(a)$ could be ϵ ,



It is an easy induction on $|w|$ to show that $\hat{\gamma}(q_0, w) = \hat{\delta}(q_0, h(w))$. Since the accepting states of A and B are the same, B accepts w if and only if A accepts $h(w)$. Put another way, B accepts exactly those strings w that are in $h^{-1}(L)$.

□

NP-Hard Problem

- It is not possible to prove that a problem P is in NP.
- Every problem in NP is reducible to P in polynomial time.

Mealy Machine and Moore Machine

- A **Mealy machine** is a finite-state machine in which the **output depends on both the current state and the current input**.
- The transition function is $\delta : Q \times \Sigma \rightarrow Q$
- The output function is $\lambda : Q \times \Sigma \rightarrow \Delta$
- A **Moore machine** is a finite-state machine in which the **output depends only on the current state**, not on the current input.
- The transition function is $\delta : Q \times \Sigma \rightarrow Q$
- The output function is $\lambda : Q \rightarrow \Delta$

- Section 4.2 and 4.3 of Chapter 4.
- Section 4.3.1—complexity analysis for different conversion. Read by yourself.